

Assignment no.6

Title: Implement a stack to manage undo operations in a text editor.
Each action (insert/delete) is pushed onto the stack, and
undo functionality pops the last action to revert changes.

Name: Diya Ravindra Bhansali

PRN: 125B1F271

Class: SY BTech IT

Source Code:

```
/*
```

```
Implement a stack to manage undo operations in a text editor.  
Each action (insert/delete) is pushed onto the stack, and  
undo functionality pops the last action to revert changes.
```

```
*/
```

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
const int max_size = 100;
```

```
class Stack
```

```
{  
    string stack1[max_size];  
    int top;
```

```
public:
```

```
    Stack()
```

```
{
```

```
    top = -1;  
}
```

```
bool isFull()  
{  
    return top == max_size - 1;  
}
```

```
bool isEmpty()  
{  
    return top == -1;  
}
```

```
void push(string x)  
{  
    if (isFull())  
    {  
        cout << "The Stack is Full!!\n";  
        return;  
    }  
    top++;  
    stack1[top] = x;  
}
```

```
string pop()  
{  
    if (isEmpty())  
    {  
        cout << "The Stack is Empty!!\n";
```

```

        return " ";
    }
    string x = stack1[top];
    top--;
    return x;
}
string display()
{
    if (isEmpty())
    {
        return "Stack is empty";
    }
    return stack1[top];
}
};

```

```

int main()
{
    int ch;
    Stack s1;
    string st;
    do
    {
        cout << "\n=====";
        cout << "\n    TEXT EDITOR UNDO OPERATIONS";
        cout << "\n=====";
        cout << "\n1. Insert";
        cout << "\n2. Delete";
        cout << "\n3. Undo";
    }
    while (ch != 0);
}

```

```
cout << "\n4. Display";
cout << "\n5. Exit";
cout << "\n\nEnter your Choice: ";
cin >> ch;
switch (ch)
{
case 1:
{
    string text;
    cout << "Enter the text: ";
    cin >> text;
    st+=text;
    s1.push(st);
    break;
}
case 2:
{
    int start;
    string text;
    cout << "Enter the text to delete: ";
    cin >> text;
    start = st.find(text);
    st.erase(start,text.length());
    s1.push(st);
    break;
}
case 3:
{
    s1.pop();
```

```

    }
    case 4:
    {
        string d = s1.display();
        cout<<"\nText Editor\n\n"<<d<<"\n\nEnd of Text Editor\n";
        break;
    }
    case 5:
        exit(0);
    default:
        cout << "\nInvalid Choice!!\n";
    }
} while (ch != 5);
}

```

Output:

```
/*
```

```
=====
```

TEXT EDITOR UNDO OPERATIONS

```
=====
```

1. Insert
2. Delete
3. Undo
4. Display
5. Exit

Enter your Choice: 1

Enter the text: PCCOE

TEXT EDITOR UNDO OPERATIONS

1. Insert
2. Delete
3. Undo
4. Display
5. Exit

Enter your Choice: 1

Enter the text: IT

TEXT EDITOR UNDO OPERATIONS

1. Insert
2. Delete
3. Undo
4. Display
5. Exit

Enter your Choice: 1

Enter the text: Department

TEXT EDITOR UNDO OPERATIONS

1. Insert
2. Delete
3. Undo
4. Display
5. Exit

Enter your Choice: 1

Enter the text: SY

TEXT EDITOR UNDO OPERATIONS

1. Insert
2. Delete
3. Undo
4. Display
5. Exit

Enter your Choice: 1

Enter the text: BTech

TEXT EDITOR UNDO OPERATIONS

1. Insert
2. Delete
3. Undo
4. Display
5. Exit

Enter your Choice: 4

Text Editor

PCCOEITDepartmentSYBTech

End of Text Editor

TEXT EDITOR UNDO OPERATIONS

1. Insert
2. Delete
3. Undo
4. Display
5. Exit

Enter your Choice: 2

Enter the text to delete: IT

TEXT EDITOR UNDO OPERATIONS

1. Insert
2. Delete
3. Undo
4. Display
5. Exit

Enter your Choice: 4

Text Editor

PCCOEDepartmentSYBTech

End of Text Editor

TEXT EDITOR UNDO OPERATIONS

-
-
1. Insert
 2. Delete
 3. Undo
 4. Display
 5. Exit

Enter your Choice: 3

Text Editor

PCCOEITDepartmentSYBTech

End of Text Editor

TEXT EDITOR UNDO OPERATIONS

1. Insert
2. Delete
3. Undo
4. Display
5. Exit

Enter your Choice: 5

Process returned 0 (0x0) execution time : 57.452 s

Press any key to continue.

*/

Code Analysis Report

Stack-Based Undo System for a Text Editor (C++)

Prepared for: Diya, PCCOE

1. Overview

The program implements a menu-driven text editor that supports Insert, Delete, Undo, and Display operations. A custom, array-backed Stack class (capacity 100) is used to track the history of the text: after every Insert or Delete, the resulting string is pushed onto the stack, and Undo pops the most recent entry so that Display shows the previous version.

The design follows the classic “snapshot stack” approach to undo — conceptually sound and easy to reason about — but the implementation has one critical correctness bug and several efficiency and robustness gaps, detailed below.

2. Functional Walkthrough

- Insert (case 1): reads a token, appends it to the running string `st`, and pushes the new `st` onto the stack.
- Delete (case 2): reads a token, finds its first occurrence in `st`, erases it in place, and pushes the resulting `st` onto the stack.
- Undo (case 3): pops the stack once, discarding the returned value, then falls through into Display.
- Display (case 4): prints whatever string currently sits on top of the stack — not `st` directly.

Because Display reads from the stack while Insert/Delete mutate `st` independently, the two are only in sync as long as no Undo has happened. This coupling is the root cause of Issue 1 below.

3. Issues Identified

#	Issue	Where	Impact
1	State-desync bug: undo pops the stack but never restores <code>st</code>	case 3 (Undo)	Critical — undo appears to work on Display, but the next Insert/Delete silently operates on the pre-undo text, corrupting the document
2	Missing break after case 3	<code>switch(ch)</code> , case 3	Fall-through into case 4 (harmless here, but fragile and easy to break with future edits)
3	No bounds check on <code>erase()</code> when text isn't found	case 2 (Delete)	<code>find()</code> returns <code>string::npos</code> ; <code>erase(npos, len)</code> throws <code>std::out_of_range</code> and crashes the program
4	<code>cin >> text</code> cannot read spaces	case 1 & case 2	Multi-word insert/delete text (e.g. “Computer Engg”) is silently truncated at the first space

#	Issue	Where	Impact
5	Full snapshot pushed on every action	push(st) in case 1 & 2	O(L) time and memory per action instead of O(1); wasteful for long documents
6	Fixed-size array (max_size = 100)	class Stack, stack1[max_size]	Hard ceiling of 100 actions; isFull() only prevents overflow, doesn't grow
7	push(string x) takes by value	class Stack::push	One extra string copy per call versus const string&
8	No redo capability	overall design	A second stack (or richer command objects) would let undone actions be reapplied

3.1 Critical: Undo does not restore program state

Trace through the sample run in the source comments: after Delete removes "IT", st becomes "PCCOEDepartmentSYBTech" and that string is also pushed to the stack. Undo then calls s1.pop(), which discards the top entry and leaves the stack's new top as the pre-delete string — so Display correctly shows "PCCOEITDepartmentSYBTech" again. However, st itself was never touched by pop(); it still equals "PCCOEDepartmentSYBTech". If the user now chooses Insert, the new text is appended to the wrong (post-delete) string, and the stack ends up holding a value that no longer matches the true edit history.

```
case 3:
{
    s1.pop();           // discards top, but 'st' is never reassigned
}
// no break -> falls into case 4, which reads the stack, not 'st'
```

Fix: pop() should return the restored string and Undo should assign it back to st:

```
case 3:
{
    if (!s1.isEmpty()) {
        s1.pop();           // discard current state
        st = s1.display();  // 'st' now matches the new top
    }
    break;
}
```

4. Time & Space Complexity (Current Implementation)

Operation	Time Complexity	Space Complexity
push() / pop()	O(1) amortized (array index only)	O(1) auxiliary
Insert action (push(st))	O(L) — copies the whole current string	O(L) per snapshot stored
Delete action (find + erase + push)	O(L) for find/erase + O(L) for the snapshot copy	O(L) per snapshot stored

Operation	Time Complexity	Space Complexity
Undo (pop)	$O(1)$ to pop, but effectively $O(L)$ because st is never resynced (see Issue 1)	—
Overall after k actions on text of avg length L	$O(k \cdot L)$ time across the session	$O(k \cdot L)$ space for the stack (every full string retained)

The dominant cost is that every Insert and Delete pushes a full copy of the current text (length L) rather than a small description of the change. For k actions on a document that grows to length L , total memory used by the stack is $O(k \cdot L)$ — e.g. 100 edits on a 1,000-character document could retain on the order of 100,000 characters of history.

5. Recommended Optimizations

5.1 Store commands, not snapshots

Replace “push the whole string” with a small Action record: {type, text, position}. Undo then reverses that one action (delete what was inserted, or re-insert what was deleted) instead of restoring a full copy of the text.

```
struct Action {
    char type;        // 'I' for insert, 'D' for delete
    string text;
    int position;
};

// Insert:  push({'I', text, st.size()}); st += text;
// Delete:  push({'D', text, start});    st.erase(start, text.size());

// Undo:
// if last.type == 'I' -> st.erase(last.position, last.text.size());
// if last.type == 'D' -> st.insert(last.position, last.text);
```

This drops per-action space from $O(L)$ to $O(1)$ (relative to the text length) and removes the st/stack desync entirely, since Undo now mutates st directly instead of reading a separate structure.

5.2 Guard against “text not found” on Delete

```
start = st.find(text);
if (start == string::npos) {
    cout << "\nText not found!\n";
    break;
}
st.erase(start, text.length());
```

5.3 Read multi-word input

cin >> text stops at whitespace. Switch to getline (after clearing the newline left by cin >> ch) so phrases like “Computer Engg” insert correctly.

```
cin.ignore();
cout << "Enter the text: ";
getline(cin, text);
```

5.4 Replace the fixed array with a dynamic container

Swap `string stack1[100]` for a `std::vector<string>` (or `std::stack<Action>` once 5.1 is applied). This removes the artificial 100-action ceiling and lets the underlying storage grow only as needed, at the usual amortized $O(1)$ push cost of a dynamic array.

5.5 Pass by const reference

```
void push(const string& x)    // was: void push(string x)
```

Avoids an unnecessary string copy on every call — minor here, but good practice once actions are pushed frequently.

5.6 Add a redo stack (optional enhancement)

Maintain a second stack of undone actions. Undo pops from the main stack and pushes onto the redo stack; a new Insert/Delete clears the redo stack (the conventional editor behavior). This is a natural extension once actions are represented as small command objects rather than full-text snapshots.

6. Complexity After Optimization

Operation	Time Complexity	Space Complexity
<code>push()/pop()</code> (command objects)	$O(1)$	$O(1)$ per action, independent of L
Insert action	$O(1)$ amortized to record; $O(L)$ only if the action is later replayed	$O(1)$ per command
Delete action	$O(L)$ to locate text once (unavoidable), $O(1)$ to record	$O(1)$ per command
Undo/Redo	$O(L)$ to reverse a single command (insert/erase substring)	$O(1)$ extra (moves command between two stacks)
Overall after k actions	$O(L)$ worst case per undo instead of $O(k \cdot L)$ cumulative	$O(k)$ space — one small command record per action

7. Summary

- Correctness: fix the Undo state-desync (Issue 1) first — it silently corrupts edit history and is the most important change.
- Robustness: guard the Delete path against “text not found” and support multi-word input.
- Efficiency: move from full-string snapshots to a command-based (delta) undo stack, cutting both time and space from $O(k \cdot L)$ to $O(k)$ plus a bounded $O(L)$ per individual undo.
- Flexibility: replace the fixed 100-slot array with a dynamic container, and consider adding a redo stack once actions are command objects.